

Module1

1. Explain the structure of a C++ program with a neat diagram. Describe the role of preprocessor directives, main() function, and user-defined functions.
2. Discuss the C++ character set and tokens in detail. Explain different types of tokens such as keywords, identifiers, constants, strings, and operators with examples.
3. Explain various data types in C++. Classify them into fundamental, derived, and user-defined data types with suitable examples.
4. What is type conversion in C++? Explain implicit and explicit type conversion with example programs.
5. Explain preprocessor directives in C++. Describe #include, #define, #undef, #if, #ifdef, and #ifndef with examples.
6. What is a namespace in C++? Explain the need for namespaces and illustrate the use of using keyword with an example.
7. Explain C++ input and output streams. Describe cin, cout, cerr, and clog along with stream manipulators such as endl, setw, setprecision, and setfill.
8. Explain dynamic memory allocation in C++. Describe the use of new and delete operators with suitable examples.
9. Discuss different control statements in C++. Explain selection, iteration, and jump statements with examples.
10. Explain function overloading in C++. What are its advantages? Write a program to demonstrate function overloading.
11. What are inline functions? Explain their advantages and disadvantages with a suitable example.
12. Explain default arguments in C++. Write a C++ program to demonstrate the use of default function arguments.
13. Explain pass by reference in C++. Compare pass by value and pass by reference with examples.
14. What is return by reference? Explain how it works in C++ with a suitable example and advantages.
15. Explain the scope of variables in C++. Discuss local, global, and block scope with examples.
16. Explain storage classes in C++. Describe auto, register, static, and extern storage classes with suitable examples.
- 17.

MODULE2

Q1.

Explain Classes and Objects in C++.

Describe how class members are accessed using access specifiers.

Demonstrate with a suitable C++ program.

Q2.

What is a Constructor in C++?

Explain the types of constructors with examples:

- Default constructor
- Parameterized constructor
- Copy constructor

Q3.

Explain the role of a Destructor in C++.

Discuss its characteristics and necessity.

Illustrate with a C++ program.

Q4.

Explain Objects as Function Arguments in C++.

Discuss the following methods with examples:

- Passing objects by value
- Passing objects by reference

Q5.

What is meant by Returning Objects from Functions?

Explain its advantages and demonstrate with a suitable C++ program.

Q6.

Differentiate between Structures and Classes in C++.

Explain memory allocation for objects in a class using new and delete.

Q7.

Explain Static Data Members and Static Member Functions in C++.

Discuss their scope, lifetime, and advantages with an example.

Q8.

Explain how Member Functions are defined outside the class using the scope resolution operator (::).
Illustrate with a suitable example.

Q9.

Define Operator Overloading.
Explain the restrictions on operator overloading in C++.
Give suitable examples.

Q10.

Explain how operator functions can be implemented as class member functions.
Demonstrate with an example program.

Q11.

Explain Overloading of Unary and Binary Operators in C++.
Write C++ programs to demonstrate both cases.

Q12.

Explain Data Conversion in Operator Overloading.
Discuss:

- Basic to Class
 - Class to Basic
 - Class to Class
- with suitable examples.

Q13.

Write a C++ program to overload the + operator to add two objects of a class.
Explain the working of the program.

Q14.

Write a C++ program to overload the ++ operator (pre or post increment).
Explain the difference between both forms.

Q15.

Discuss the importance of Operator Overloading in real-world applications.
Explain with suitable examples.

MODULE3

Inheritance

1. Explain Inheritance in C++.
Define Base class and Derived class. Write a C++ program to demonstrate inheritance using suitable data members and member functions.
2. Explain Access Specifiers in Inheritance (public, protected, private).
Show how base class members are accessed in derived classes with a neat example.
3. Explain the different Types of Inheritance in C++.
Describe Single, Multiple, Multilevel, Hierarchical, and Hybrid inheritance with diagrams and examples.
4. Differentiate between Public and Private Inheritance in C++.
Explain how visibility of base class members changes in each case with example code.
5. Explain Constructors and Destructors in Derived Classes.
Describe the order of execution of constructors and destructors in inheritance with a program.
6. Explain Aggregation in C++.
How is it different from inheritance? Demonstrate aggregation using a real-world example.
7. What is a Virtual Function?
Explain its need in inheritance and demonstrate how it supports runtime polymorphism.

Polymorphism

8. Define Polymorphism in C++.
Explain compile-time and runtime polymorphism with suitable examples.
9. Explain the Concept of Virtual Functions and Late Binding.
How does late binding help achieve runtime polymorphism? Illustrate with a program.
10. What is an Abstract Class?
Explain pure virtual functions and show how abstract classes are implemented in C++.
11. Explain Virtual Destructors.
Why are they required in inheritance? Write a program to justify their use.
12. What is a Virtual Base Class?
Explain the diamond problem and show how virtual base classes solve it with a program.

Other OOP Concepts Related to Polymorphism

13. Explain Friend Function and Static Member Function in C++.
Write a program demonstrating both and mention their advantages.
14. Explain Assignment and Copy Initialization in C++.
Differentiate between them with suitable examples.
15. Explain the this Pointer in C++.
State its uses and demonstrate it using a program.
16. What are Concrete Classes?
Explain their role in object-oriented programming with example.
17. Explain Polymorphism and its Roles in C++.
Discuss how inheritance, virtual functions, and base class pointers enable polymorphism.

18. With a suitable C++ program, explain how polymorphism is achieved using inheritance and virtual functions.
Include base class pointer and derived class objects.
19. Explain the relationship between Inheritance, Virtual Functions, Abstract Classes, and Polymorphism with a neat program and explanation.
20. Design a C++ program demonstrating:
 - Base and derived classes
 - Virtual function
 - Late binding
 - Virtual destructorExplain the output.

Module4

1. Explain Function Templates in C++.
Demonstrate a function template with multiple arguments using a suitable example.
2. What are Class Templates?
Explain their syntax and advantages. Write a C++ program to demonstrate a class template.
3. Explain Templates and Inheritance.
Show how a class template can be inherited by another class with a suitable program.
4. Differentiate between Function Templates and Class Templates.
Explain with examples.

Section B – Exception Handling

5. Explain Exception Handling in C++.
Describe the keywords try, catch, and throw with a suitable program.
6. Explain the Use of Exception Handling in C++.

How does it improve program reliability? Demonstrate handling of multiple exceptions.

7. What are Standard Exceptions in C++?
Explain any four standard exception classes with examples.

Section C – File Handling

8. Explain File Handling Concepts in C++.
Describe file streams (ifstream, ofstream, fstream) and file modes with example.
9. Write a C++ program to perform file operations such as:
 - Writing data into a file
 - Reading data from a fileExplain the program.